

Experiment- 7

Long Short-Term Memory (LSTM) for Sentiment Analysis

1. Aim

To study Long Short-Term Memory (LSTM) based sentiment analysis by modelling longer-term dependencies in text through training an LSTM on a subset of the IMDB dataset with short training epochs, and to evaluate performance using learning curves and confusion matrix, along with a brief comparison to a simple RNN.

2. Theory

Introduction to Sentiment Analysis

Sentiment Analysis is a supervised Natural Language Processing (NLP) task that aims to automatically identify and classify the emotional polarity expressed in textual data. The most common form of sentiment analysis focuses on determining whether a given text conveys a positive or negative sentiment. This task plays a crucial role in applications such as opinion mining, customer feedback analysis, recommendation systems, market analysis, and social media monitoring.

In this experiment, sentiment analysis is formulated as a binary classification problem, where the objective is to predict the sentiment label of a movie review based on its textual content:

$$y \in \{0,1\}$$

where,

- 0 represents a *Negative review*,
- 1 represents a *Positive review*.

Since textual data is inherently sequential, capturing word order and contextual dependencies is essential for accurate sentiment prediction. Recurrent Neural Networks (RNNs) and their advanced variants, such as Long Short-Term Memory (LSTM) networks, are particularly well-suited for this task.

Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) is a specialized type of Recurrent Neural Network (RNN) designed to mitigate the vanishing gradient problem in traditional RNNs and improve the learning of long-term dependencies in sequential data. LSTM networks were introduced by Hochreiter and Schmidhuber in 1997 with the explicit goal of enabling neural networks to learn and retain long-term dependencies in sequential data.

Unlike standard RNNs, which rely solely on a single hidden state, LSTMs introduce an internal cell state that acts as a memory pipeline. This cell state allows information to flow across time steps with minimal modification, making it easier for gradients to propagate during backpropagation through time (BPTT). As a result, LSTMs are capable of remembering important contextual information over long sequences while selectively forgetting irrelevant details.

LSTM Cell Components-

An LSTM cell consists of several interacting components that regulate the flow of information using gating mechanisms. These gates are implemented using sigmoid and tanh activation functions, which enable fine-grained control over memory updates.

- **Forget Gate (f_t):** The forget gate determines which information from the previous cell state should be retained or discarded. This process is implemented by a sigmoid activation function.

The decision is based on the previous hidden state and the current input. The output of the forget gate is a vector of values between 0 and 1, where values close to 0 indicate information to be forgotten, and values close to 1 indicate information to be retained.

$$f_t = \sigma(W_f[h_{t-1}, p_t] + b_f)$$

- **Input Gate (i_t):** The input gate controls the extent to which new information from the current input should be written to the cell state. This gate protects the memory from being corrupted by irrelevant or noisy inputs and ensures that only meaningful information is incorporated.

$$i_t = \sigma(W_i[h_{t-1}, p_t] + b_i)$$

- **Candidate Cell State ($\tilde{C}_t$):** The candidate cell state represents new information generated from the current input and previous hidden state that may be added to the cell state. It is computed using a tanh activation function, producing values in the range [-1,1]. The input gate determines how much of this candidate information should be incorporated into the updated cell state.

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, p_t] + b_c)$$

- **Cell State Update (C_t):** The cell state update combines the retained information from the previous cell state with the newly generated candidate information. This additive update mechanism is a key innovation of LSTM networks, as it allows gradients to flow across long sequences without rapid decay, enabling effective learning of long-term dependencies.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

- **Output Gate (o_t):** The output gate determines which part of the updated cell state should be exposed as the hidden state for the current time step. This allows the LSTM to control how much of its internal memory influences the output.

$$o_t = \sigma(W_o[h_{t-1}, p_t] + b_o)$$

- **Hidden State (h_t):** The hidden state represents the final output of the LSTM cell at time step t . It is obtained by applying a tanh activation to the updated cell state and modulating it with the output gate.

$$h_t = o_t \cdot \tanh(C_t)$$

LSTM Architecture & Information Flow-

The neural network architecture of an LSTM block, illustrated in Figure 1, demonstrates how LSTM networks extend the memory capabilities of traditional RNNs. In addition to the hidden state used in RNNs, an LSTM block introduces additional components, including the cell state (C_t), forget gate (f_t), input gate (i_t), candidate cell state ($\tilde{C}_t$), and output gate (o_t). These components interact in a carefully designed manner to regulate the flow of information across time steps.

The p_t , h_{t-1} , and C_{t-1} correspond to the input of the current time step, the hidden output from the previous time step, and the cell state (memory) of the previous unit, respectively. The information from the previous LSTM unit is combined with the current input to effectively model sequential patterns and contextual relationships in text data. The LSTM blocks are mainly divided into three gates: forget, input, and output. Each of these gates is connected to the cell state to provide the necessary information that flows from the current time step to the next.

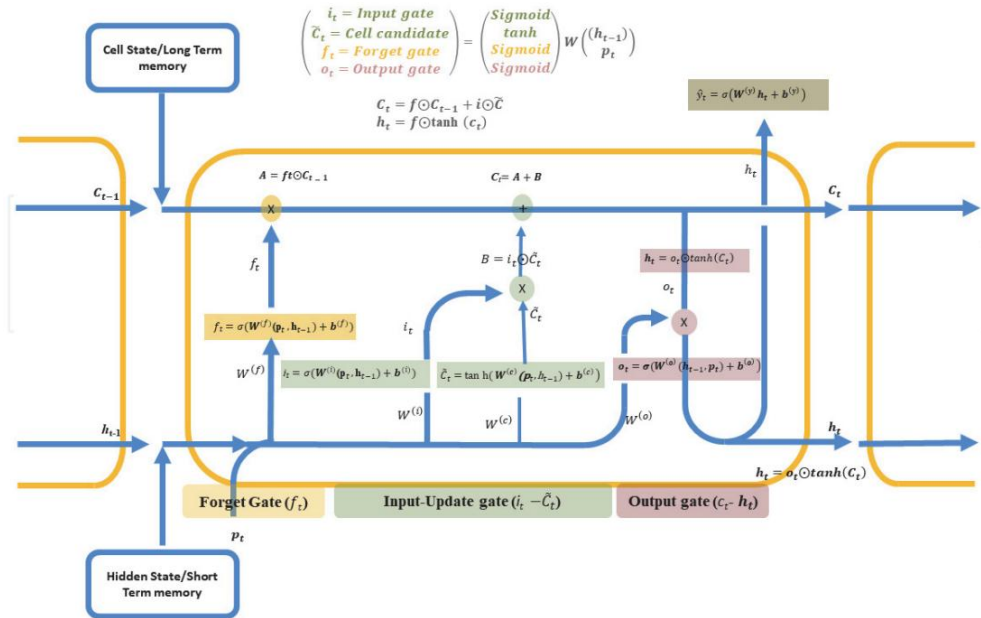


Figure 1- Architecture of LSTM

(Source: H. Okut, "Deep Learning for Subtyping and Prediction of Diseases: Long Short-Term Memory," IntechOpen, 2021)

Due to these properties, LSTMs are widely used in applications such as sentiment analysis, machine translation, speech recognition, and time-series forecasting, where understanding long-range dependencies is critical for accurate predictions.

Mechanism for Mitigating the Vanishing Gradient Problem in LSTMs-

A key reason why LSTMs are able to overcome the vanishing gradient problem lies in the structure of the Cell State Update (C_t) equation:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

Unlike traditional RNNs, where the hidden state is repeatedly transformed by nonlinear activation functions (such as tanh or sigmoid), leading to rapid gradient shrinkage, the LSTM cell state follows an additive update mechanism.

This has the following two important effects:

- The term $f_t \cdot C_{t-1}$ allows the previous memory to pass forward almost unchanged when the forget gate value f_t is close to 1.
- Since the update is additive rather than multiplicative across many time steps, gradients can flow backward through time without repeatedly being multiplied by small values, thus preventing them from vanishing.

Thus, in simpler terms:

- In traditional RNNs, repeated multiplicative transformations across time steps lead to rapid attenuation of gradients.
- In LSTMs, the additive cell state updates combined with gated mechanisms enable stable gradient propagation over long sequences.

The cell state, therefore, acts as an efficient pathway for gradient flow, allowing information and gradients to propagate across long sequences with minimal attenuation.

Merits of Long Short-Term Memory

- **Handles Long-Term Dependencies:** LSTM networks are capable of learning long-term dependencies in sequential data, overcoming the vanishing gradient problem present in traditional RNNs.
- **Effective Memory Management:** The use of forget, input, and output gates allows LSTM to selectively store, update, or discard information, leading to better sequence modelling.
- **Suitable for Sequential Data:** LSTMs perform well on time-series, text, speech, and sentiment analysis tasks where data has temporal dependencies.
- **Stable Training:** Due to controlled gradient flow through the cell state, LSTMs provide more stable training compared to simple RNNs.
- **Better Performance on Contextual Tasks:** LSTMs capture contextual information over longer sequences, improving performance in tasks such as language modelling and text classification.

Demerits of Long Short-Term Memory

- **High Computational Cost:** LSTM networks involve multiple gate computations, making them computationally expensive compared to standard RNNs.
- **Longer Training Time:** Due to their complex structure, LSTMs require more time to train, especially on large datasets.
- **Large Memory Requirement:** The presence of multiple weight matrices and gates increases memory consumption.
- **Risk of Overfitting on Small Datasets:** When trained on small datasets, LSTMs may overfit if proper regularization techniques are not applied.
- **Complex Architecture:** The internal structure of LSTM cells makes them harder to understand, tune, and debug compared to simpler models.

3. Code & Results:

Step 1: Import Libraries

INPUT:

Importing Libraries

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import re, string, os
```

```
from tensorflow.keras.models import Sequential, load_model
```

```
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, Dense, Input
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
from tensorflow.keras.optimizers import Adam
```

```
from tensorflow.keras.callbacks import ModelCheckpoint
```

```
from tensorflow.keras.callbacks import ReduceLROnPlateau
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import roc_curve, auc, precision_recall_curve, average_precision_score
from matplotlib.ticker import FormatStrFormatter
print("All the required libraries are imported.\n")
```

OUTPUT:

All the required libraries are imported.

Step 2: Define Parameters

INPUT:

```
# Defining paths & Parameters
```

```
VOCAB_SIZE = 10000
MAX_LEN = 300
EMBED_DIM = 256
RNN_UNITS = 256
LSTM_UNITS = 64
BATCH_SIZE = 128
EPOCHS = 150
BASE_PATH = "/kaggle/input/standford-dataset/aclImdb"
TRAIN_POS = os.path.join(BASE_PATH, "train/pos")
TRAIN_NEG = os.path.join(BASE_PATH, "train/neg")
TEST_POS = os.path.join(BASE_PATH, "test/pos")
TEST_NEG = os.path.join(BASE_PATH, "test/neg")
```

```
print(f"Parameters for training are set as:")
print(f" Vocabulary Size: {VOCAB_SIZE}")
print(f" Maximum Sequence Length: {MAX_LEN}")
print(f" Embedding Dimensions: {EMBED_DIM}")
print(f" RNN units: {RNN_UNITS}")
print(f" LSTM units: {LSTM_UNITS}")
print(f" Batch Size: {BATCH_SIZE}")
print(f" Epochs: {EPOCHS}")
```

OUTPUT:

Parameters for training are set as:

Vocabulary Size: 10000

Maximum Sequence Length: 300

Embedding Dimensions: 256

RNN units: 256

LSTM units: 64

Batch Size: 128

Epochs: 150

Step 3: Dataset Preprocessing

INPUT:

Dataset Preprocessing

```
def clean_text(text):  
    text = text.lower()  
    text = re.sub(r'\[.*?\]', '', text)  
    text = re.sub(r'%s' % re.escape(string.punctuation), '', text)  
    text = re.sub(r'\w*\d\w*', '', text)  
    text = re.sub(r'\s+', ' ', text).strip()  
    return text
```

Load Reviews

```
def load_reviews(folder, label, max_samples):  
    texts, labels = [], []  
    files = sorted(os.listdir(folder))[:max_samples]  
    for f in files:  
        with open(os.path.join(folder, f), encoding="utf-8") as file:  
            texts.append(file.read())  
            labels.append(label)  
    return texts, labels
```

OUTPUT:

✓ Preprocessing functions defined successfully

Step 4: Train-Val-Test Splitting

INPUT:

Train-Val-Test Splitting

```
train_pos, y_train_pos = load_reviews(TRAIN_POS, 1, 4250)
```

```

train_neg, y_train_neg = load_reviews(TRAIN_NEG, 0, 4250)
X_train_full = np.array(train_pos + train_neg)
y_train_full = np.array(y_train_pos + y_train_neg)
test_pos, y_test_pos = load_reviews(TEST_POS, 1, 750)
test_neg, y_test_neg = load_reviews(TEST_NEG, 0, 750)
X_test = np.array(test_pos + test_neg)
y_test = np.array(y_test_pos + y_test_neg)

print("Train+Val:", len(X_train_full))
print("Test:", len(X_test))

X_train_full = np.array([clean_text(x) for x in X_train_full])
X_test = np.array([clean_text(x) for x in X_test])
X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full, test_size=1500,
                                                  random_state=42, stratify=y_train_full)

print("Train:", len(X_train))
print("Val:", len(X_val))
print("Test:", len(X_test))
print("Train balance:", np.bincount(y_train))
print("Val balance:", np.bincount(y_val))
print("Test balance:", np.bincount(y_test))

# Tokenization
tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token="<UNK>")
tokenizer.fit_on_texts(X_train)
X_train_pad = pad_sequences(tokenizer.texts_to_sequences(X_train), maxlen=MAX_LEN)
X_val_pad = pad_sequences(tokenizer.texts_to_sequences(X_val), maxlen=MAX_LEN)
X_test_pad = pad_sequences(tokenizer.texts_to_sequences(X_test), maxlen=MAX_LEN)

```

OUTPUT:

Train+Val: 8500

Test: 1500

Train: 7000

Val: 1500

Test: 1500

Train balance: [3500 3500]

Val balance: [750 750]

Test balance: [750 750]

Step 5: Model Architectures & Checkpoints

INPUT:

Defining Model Architectures & Checkpoints

Simple RNN Model

```
model_rnn = Sequential([
    Input(shape=(MAX_LEN,)),
    Embedding(VOCAB_SIZE, EMBED_DIM),
    SimpleRNN(RNN_UNITS, dropout=0.3, recurrent_dropout=0.1),
    Dense(1, activation='sigmoid')])
model_rnn.compile( optimizer=Adam(3e-5), loss='binary_crossentropy', metrics=['accuracy'] )
```

LSTM Model

```
model_lstm = Sequential([
    Input(shape=(MAX_LEN,)),
    Embedding(VOCAB_SIZE, EMBED_DIM),
    LSTM(LSTM_UNITS, dropout=0.3),
    Dense(1, activation='sigmoid') ])
model_lstm.compile( optimizer=Adam(1e-5), loss='binary_crossentropy', metrics=['accuracy'] )
```

Model Checkpoints

```
rnn_ckpt = ModelCheckpoint("best_rnn.keras", monitor="val_accuracy",
                           save_best_only=True, mode='max', verbose=0)
lstm_ckpt = ModelCheckpoint("best_lstm.keras", monitor="val_accuracy",
                            save_best_only=True, mode='max', verbose=0)
```

```
reduce_lr_rnn = ReduceLROnPlateau( monitor="val_loss", factor=0.5, patience=6, min_lr=1e-6,
                                     verbose=0 )
```

```
reduce_lr_lstm = ReduceLROnPlateau( monitor="val_loss", factor=0.5, patience=6, min_lr=1e-6,
                                     verbose=0 )
```


OUTPUT:

✓ RNN and LSTM models defined successfully

Step 6: Model Training**INPUT:**

```
# Model Training
```

```
print("\nStarting training...\n")
```

```
print("\nTraining RNN...\n")
```

```
history_rnn = model_rnn.fit( X_train_pad, y_train, epochs=EPOCHS, batch_size=BATCH_SIZE,  
                             validation_data=(X_val_pad, y_val), callbacks=[rnn_ckpt, reduce_lr_rnn], verbose=0 )
```

```
print("\nTraining LSTM...\n")
```

```
history_lstm = model_lstm.fit( X_train_pad, y_train, epochs=EPOCHS, batch_size=BATCH_SIZE,  
                               validation_data=(X_val_pad, y_val), callbacks=[lstm_ckpt, reduce_lr_lstm], verbose=0 )
```

```
print("\nTraining completed.")
```

```
# Loading Best Models
```

```
best_rnn = load_model("best_rnn.keras")
```

```
best_lstm = load_model("best_lstm.keras")
```

```
print("\nBest models loaded for both RNN and LSTM.")
```

OUTPUT:

Starting training...

Training RNN...

Training LSTM...

Training completed.

Best models loaded for both RNN and LSTM.

Step 7: Model Evaluation**INPUT:**

```
# Model Evaluation
```

```
# Accuracy
```

```
print("\nFinal Accuracies-")
```

```
print("\n===== SIMPLE RNN =====")
```

```

print(f"Train Accuracy: {history_rnn.history['accuracy'][-1]*100:.2f}%")
print(f"Val Accuracy: {best_rnn.evaluate(X_val_pad, y_val, verbose=0)[1]*100:.2f}%")
print(f"Test Accuracy: {best_rnn.evaluate(X_test_pad, y_test, verbose=0)[1]*100:.2f}%")
print("\n===== LSTM =====")
print(f"Train Accuracy: {history_lstm.history['accuracy'][-1]*100:.2f}%")
print(f"Val Accuracy: {best_lstm.evaluate(X_val_pad, y_val, verbose=0)[1]*100:.2f}%")
print(f"Test Accuracy: {best_lstm.evaluate(X_test_pad, y_test, verbose=0)[1]*100:.2f}%")

```

Classification Report

```

print("\nFinal Classification Reports-")
print("\n===== RNN Classification Report =====")
print(classification_report(y_test, y_pred_rnn, digits=4))
print("\n===== LSTM Classification Report =====")
print(classification_report(y_test, y_pred_lstm, digits=4))

```

OUTPUT:

Final Accuracies

Simple RNN

Train Accuracy: **90.76%**

Val Accuracy: **79.47%**

Test Accuracy: **78.87%**

LSTM

Train Accuracy: **96.17%**

Val Accuracy: **87.33%**

Test Accuracy: **84.80%**

Classification Reports

RNN Classification Report

	precision	recall	f1-score	support
0	0.7954	0.7773	0.7862	750
1	0.7823	0.8000	0.7910	750
accuracy			0.7887	1500
macro avg	0.7888	0.7887	0.7886	1500
weighted avg	0.7888	0.7887	0.7886	1500

LSTM Classification Report

	precision	recall	f1-score	support
0	0.8329	0.8707	0.8514	750
1	0.8645	0.8253	0.8445	750
accuracy			0.8480	1500
macro avg	0.8487	0.8480	0.8479	1500
weighted avg	0.8487	0.8480	0.8479	1500

Step 8: Graph Visualization

INPUT: (View Plots For: RNN Only)

Graph Visualization - RNN Only

```
print("\nLearning Curves for RNN-")
```

```
plot_per_model_metrics(history_rnn, "RNN", gap=0.2)
```

```
print("\nConfusion Matrix - RNN")
```

```
plt.figure(figsize = (5,4))
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_rnn), annot=True, fmt='d', cmap='Blues')
```

```
plt.title("Confusion Matrix - RNN")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

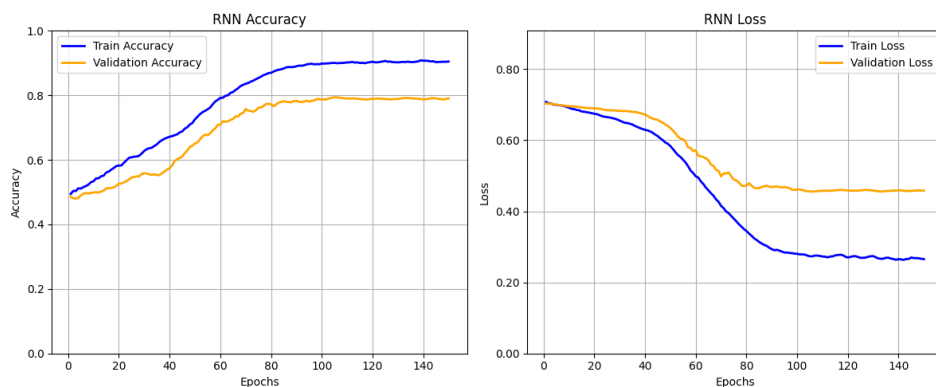
```
plt.show()
```

```
print("\nROC & PR Curves for RNN-")
```

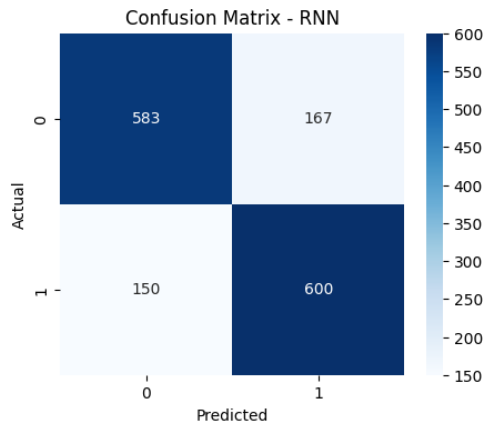
```
plot_model_roc_pr(best_rnn, "RNN", X_test_pad, y_test, X_val_pad, y_val)
```

OUTPUT:

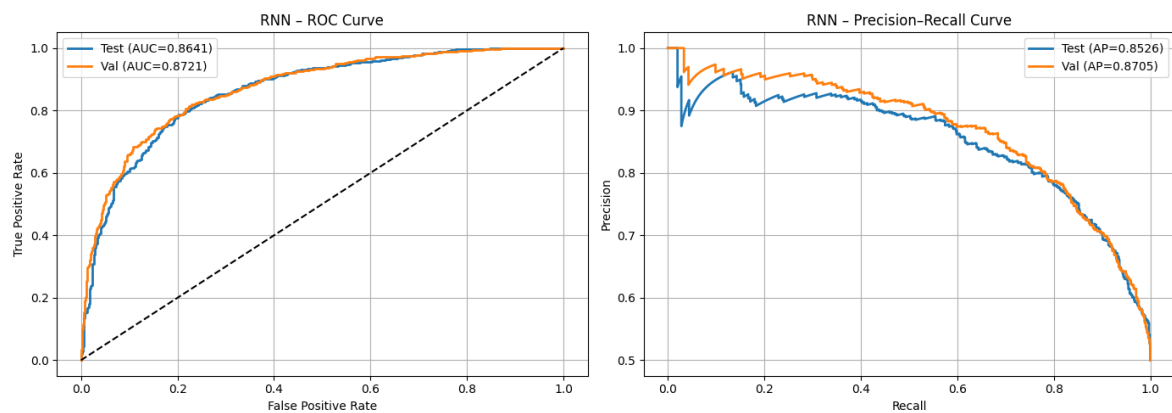
Learning Curves - RNN



Confusion Matrix - RNN



ROC & Precision-Recall Curves - RNN



INPUT: (View Plots For: LSTM Only)

Graph Visualization - LSTM Only

```
print("\nLearning Curves for LSTM-")
```

```
plot_per_model_metrics(history_lstm, "LSTM", gap=0.2)
```

```
print("\nConfusion Matrix - LSTM")
```

```
plt.figure(figsize = (5,4))
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_lstm), annot=True, fmt='d', cmap='Blues')
```

```
plt.title("Confusion Matrix - LSTM")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

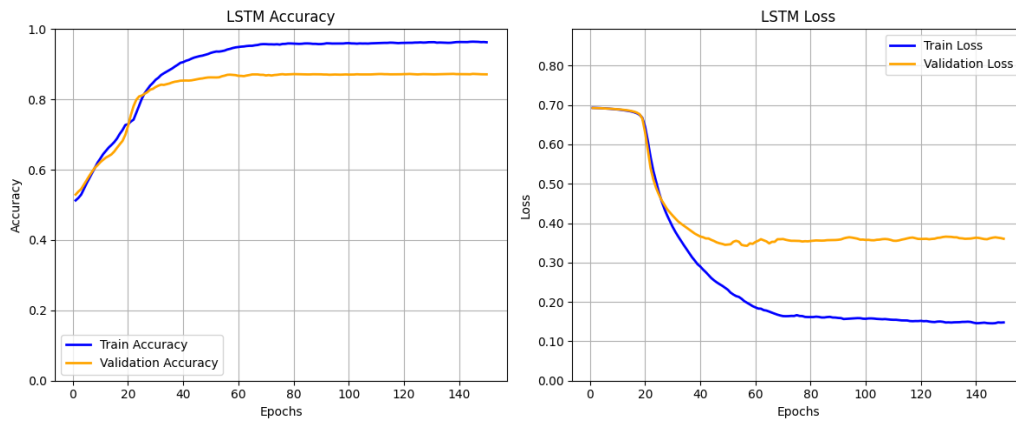
```
plt.show()
```

```
print("\nROC & PR Curves for LSTM-")
```

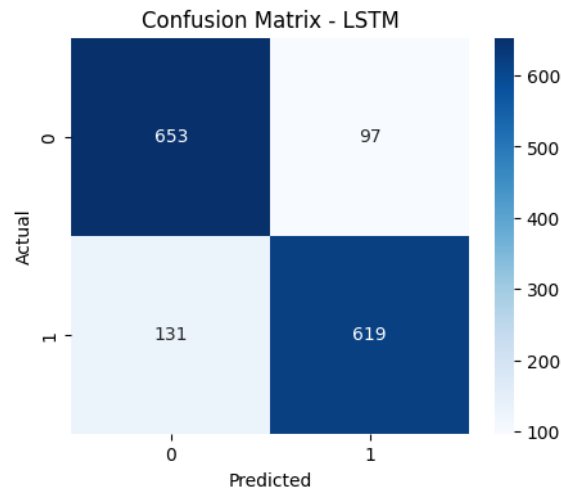
```
plot_model_roc_pr(best_lstm, "LSTM", X_test_pad, y_test, X_val_pad, y_val)
```

OUTPUT:

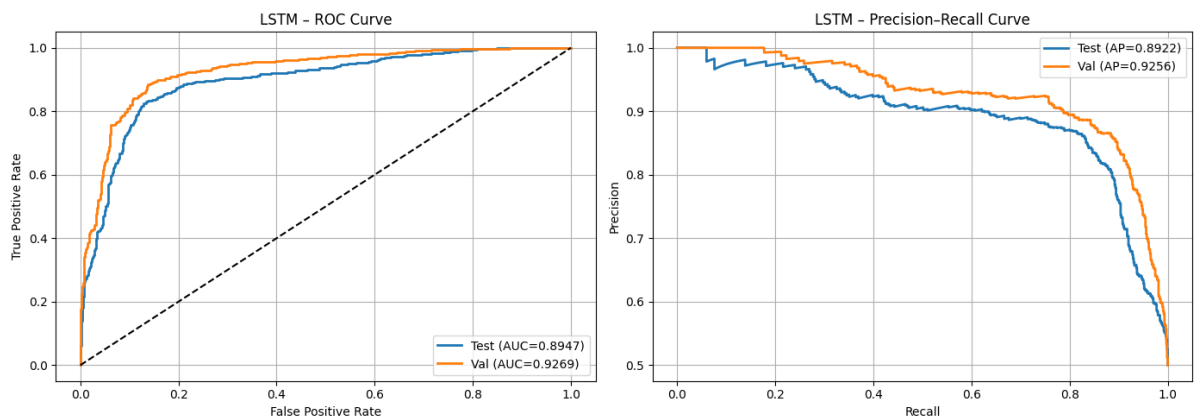
Learning Curves - LSTM



Confusion Matrix - LSTM



ROC & Precision-Recall Curves - LSTM



INPUT: (View Plots For: Combined (RNN & LSTM))

Graph Visualization - Combined (RNN vs LSTM)

```
print("\nCombined Training Curves (RNN vs LSTM)-")
```

```
plot_side_by_side_epoch_metrics(
```

```
    history_rnn, history_lstm,
```

```
    "accuracy", "loss",
```

```
    "Accuracy", "Loss",
```

```
    "Training Accuracy (RNN vs LSTM)",
```

```
    "Training Loss (RNN vs LSTM)"
```

```
)
```

```
print("\nCombined Validation Curves (RNN vs LSTM)-")
```

```
plot_side_by_side_epoch_metrics(
```

```
    history_rnn, history_lstm,
```

```
    "val_accuracy", "val_loss",
```

```
    "Accuracy", "Loss",
```

```
    "Validation Accuracy (RNN vs LSTM)",
```

```
    "Validation Loss (RNN vs LSTM)"
```

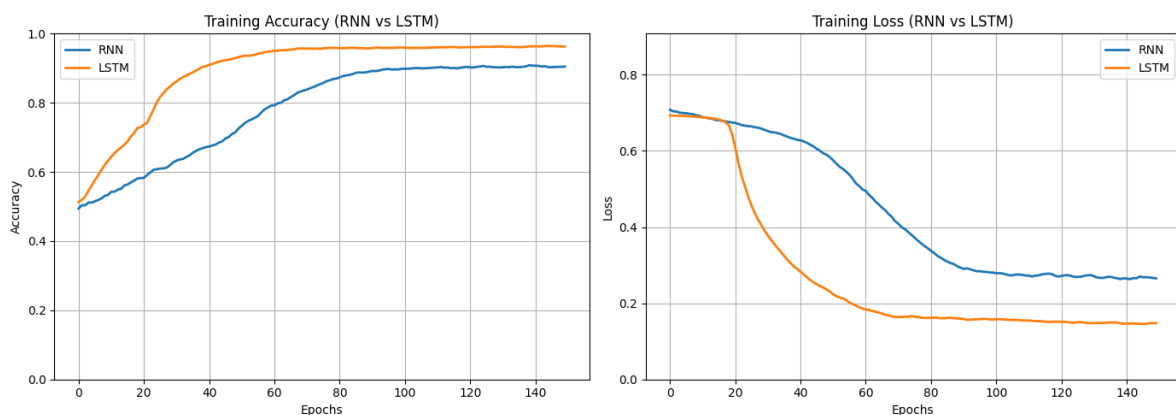
```
)
```

```
print("\nCombined ROC & PR Curves")
```

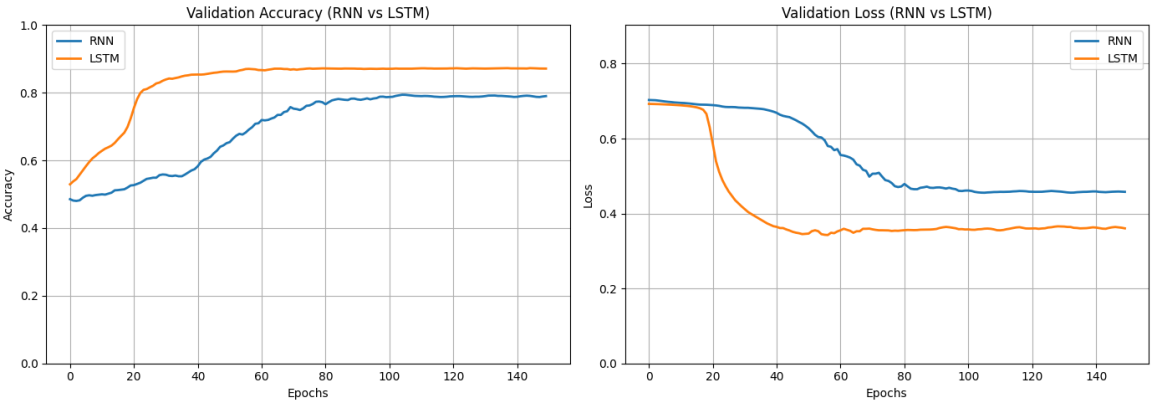
```
plot_roc_pr_curves(best_rnn, best_lstm, X_test_pad, y_test, X_val_pad, y_val)
```

OUTPUT:

Combined Training Curves (RNN vs LSTM)



Combined Validation Curves (RNN vs LSTM)



Combined ROC & Precision-Recall Curves

